OpenDesign + Smooth-Transitions + Look-and-Feel Polish — Scoping Doc

Date: 2026-06-25 **Operator directive (2026-06-25):** "Make transitions between all application's screens, fragments and views more smooth with some stunning visual effects! Do use heavily OpenDesign and fine tune and polish all existing look and feel! Cover all with additional tests!!!" **Posture:** READ-ONLY source inspection + WebFetch. No code changed, no builds, no commit. Anti-bluff (§6.J / §6.AB): every claim below cites the actual file/line read. "Fragments" = Compose screens/destinations (Lava is 100% Compose; no XML/Fragments). **Constitution anchor:** §11.4.162 (OpenDesign UI design-system mandate) — inherited via §6.AD from `constitution/`.

---

1. OpenDesign reality check (§11.4.162) — NOT integrated

Finding: OpenDesign is NOT installed or wired into Lava. Grep across `gradle/libs.versions.toml`, every `build.gradle.kts`, and `core/designsystem/` for `open-design` / `opendesign` / `nexu` returns **zero** hits in Lava's own source. The only matches are:

- `panoptic/CLAUDE.md:1496`, `panoptic/AGENTS.md:1543`, `panoptic/CONSTITUTION.md:1205` — these are a **sibling project's** copies of the §11.4.162 mandate text, not Lava wiring.
- `submodules/helixqa/...` "nexus" hits are HelixQA's `pkg/nexus` browser package — unrelated to OpenDesign.

So per §11.4.162 ("every project producing user-facing interfaces MUST use OpenDesign … install as a project dependency"), Lava is currently **non-compliant**, and OpenDesign integration is in-scope for this cycle.

What OpenDesign actually is (WebFetch — github.com/nexu-io blocked, resolved via WebSearch)

GitHub raw fetch is blocked by network policy; the facts below come from the public repo description + QUICKSTART via WebSearch:

- OpenDesign (`github.com/nexu-io/open-design`, current release **0.7.0**) is a **local-first, open-source design-systems tool**: a native desktop app + **stdio MCP server** + per-agent install scripts, shipping **142+ Design Systems**. It is consumed by AI coding agents (Claude Code, Codex, Cursor, OpenCode, Qwen, Kimi, 16+ CLIs) over **MCP**.
- A "design system" in OpenDesign is a folder of: `manifest.json`, `DESIGN.md` (canonical design prose), **`tokens.css`** (compiled CSS custom properties — the canonical token artifact), `components.html`, `assets/`, `fonts/`, preview pages. Recent versions ship a **structured `tokens.css` schema** (default + `kami`).
- **Install:** one-line per-agent — `od mcp install <agent>` — which drops `~/.config/<agent>/open-design.json` + an MCP snippet. Agents then query design systems by name (get tokens CSS, JSX components, entry HTML).

Honest integration boundary (the critical caveat)

OpenDesign is web/CSS-and-agent-oriented; it is NOT a Maven/Gradle artifact and NOT a Kotlin/Compose library. Its canonical output is `tokens.css` (CSS custom properties) consumed over MCP — there is **no `androidx`-style dependency to add to `libs.versions.toml`**. Lava is Jetpack Compose (Kotlin). Therefore "use OpenDesign" in the Android context means, concretely:

1. Install the OpenDesign MCP server for the dev agents (`od mcp install claude-code`, etc.).
2. **Adopt (or author, per §11.4.74 extend-don't-reimplement) a Lava-branded OpenDesign design system** — `tokens.css` carrying the full light+dark palette, type scale, spacing scale, component tokens, sourced from Lava brand assets (§11.4.35).
3. **Generate / sync the Compose token layer** (`AppColors`, `AppTypography`, `AppSpaces`, `AppShapes`, …) **from** that `tokens.css` so OpenDesign is the source-of-truth and the hand-coded Kotlin tokens become generated artifacts. The `tokens.css` ↔ Kotlin sync is the load-bearing bridge and needs a small codegen/check step (Wave A deliverable).

Operator input needed (see §6): whether to (a) author a bespoke Lava design system inside OpenDesign vs adopt+rebrand one of the 142 shipped systems; (b) commit `tokens.css` into the repo as the tracked source-of-truth; (c) brand-asset palette/fonts to seed it.

---

2. Current theming state — already a real token system, hand-coded, dark-theme-complete

`core/designsystem/` (`LavaTheme`). Cited files below.

- **Theme entry:** `theme/Theme.kt` — `LavaTheme(theme, isDark, isDynamic, content)`. Maps a `Theme` enum to a palette factory, builds a Material3 `lightColorScheme/darkColorScheme`, and provides `LocalColors`. (`Theme.kt:14-72`)
- **Dark theme: YES, comprehensive.** Every palette factory takes `isDark` and returns full dark + light variants; `isSystemInDarkTheme()` is the default. (`Theme.kt:17,27-37`)
- **8 named palettes + Material You:** `yoleColors` (brand default), `draculaColors`, `solarizedColors`, `nordColors`, `monokaiColors`, `gruvboxColors`, `oneDarkColors`, `tokyoNightColors`, plus `DYNAMIC` (Material You, API 31+ via `isMaterialYouAvailable()`). (`Theme.kt:21-37,74-75`)
- **Brand colors:** hand-coded Material tonal ramps in `theme/Colors.kt` — note the "Indigo*" ramp is actually the **brand red** (`Indigo40 = 0xFFB3261E`, `Indigo50 = 0xFFDE3730`); "Studio*" = purple secondary, "Lipstick*" = pink tertiary, plus custom green/red/orange/denim ramps. ~180 lines of literal `Color(0xFF…)` constants. (`Colors.kt:1-180`)
- **Token classes already exist** (this is the good news — Lava already has a design-token architecture, just not OpenDesign-sourced): `AppColors`, `AppTypography`, `AppShapes`, `AppSizes`, `AppSpaces`, `AppBorders`, `AppElevations`, surfaced through `AppTheme` object via `CompositionLocals`. (`theme/AppTheme.kt:6-41`)
- **Typography:** `AppTypography` is the full Material type scale (display/headline/title/body/label) — **but every style uses `fontFamily = FontFamily.Default`**; there is **no brand font**. (`AppTypography.kt:11-117`). No `.ttf/.otf` font assets exist in Lava (only vendored ones under `submodules/helixqa/...`).
- **Components:** ~25 design-system composables under `component/` (AppBar, Buttons, Dialog, TextField, NavigationBar, ModalBottomSheet, Placeholder, ProgressIndicator, Scaffold, Surface, Pagination, …).
- **Theming tests today:** `PaletteContractTest.kt` (Robolectric) asserts all 8 palettes produce valid light+dark colors and `primary != surface`. `LavaIconsAppIconColorRegressionTest`, `A11yContentDescriptionTest`. **No visual-regression /**

screenshot infra at all (no Paparazzi, no Roborazzi, no perceptual-diff — grep confirms none). This is the §11.4.162 "visual regression tests with per-pixel/perceptual diff" gap.

Implication: Lava does not need a token *architecture* built from scratch — it needs the existing hand-coded token classes **re-sourced from OpenDesign `tokens.css`** + a brand font + visual-regression coverage.

---

3. Current transition state — a system exists, but most destinations use `Default` (= no transition)

The transition machinery lives in `core/navigation` and is wired per-destination in `:app`.

- **Animation model:** `navigation/ui/NavigationAnimations.kt` — `NavigationAnimations(enter/exit/popEnter/popExit)` data class with presets:
  - `Default` = all-null = **NavHost default (no custom transition)**. (`NavigationAnimations.kt:29`)
  - `ScaleInOutAnimation` = scaleIn+expandIn+fadeIn / fadeOut+shrinkOut+scaleOut. (`:30-49`)
  - `FadeInOutAnimations` = fadeIn / fadeOut. (`:50-55`)
  - `slideInLeft/slideOutLeft/slideInRight/slideOutRight` helpers. (`:57-60`)
- **Wiring:** `NavigationHost.kt` plumbs these into Navigation-Compose `composable(... enterTransition=…)` / `navigation(...)`. (`NavigationHost.kt:42-56`)
- **Per-destination assignment** (`app/.../navigation/MobileNavigation.kt`): | Destination | Animation | Notes | |---|---|---| | login, credentials, credentialsManager, providerConfig | `ScaleInOutAnimation` | (`:55-68`) | | category, topic | `ScaleInOutAnimation` | (`:97,105`) | | searchInput (top-level), searchResult (top-level) | **`Default` (none)** | (`:76,88`) | | bottom-nav tabs (Search/Forum/Topics/Menu) | directional `slideIn/Out` by tab ordinal | polished — (`:329-350`) | | nested search graph: searchHistory `Default`, searchInput `FadeInOutAnimations`, searchResult `Default` | mixed | (`:178,186,194`) |
- **In-screen (non-nav) animations that DO exist:**
  - Onboarding wizard: `AnimatedContent` with **320ms `FastOutSlowInEasing` slide + 220ms `LinearOutSlowInEasing` fade** between steps (the §6.L 62nd polish). This is the current best-in-class example. (`feature/onboarding/.../OnboardingScreen.kt:109-132`)
  - Topic: `fadeIn() + slideInVertically { it }` on a sub-element. (`feature/topic/.../TopicScreen.kt:393`)

- **What's ABSENT entirely:**
  - **No shared-element / shared-bounds transitions** anywhere (no `SharedTransitionLayout` / `sharedElement` usage — grep confirms none). E.g. tapping a search-result row → topic does a scale, not a shared poster/title morph.
  - **The two highest-traffic transitions (search-input → results, results → topic at the top level) are `Default` = no animation** → abrupt cuts, exactly the "abrupt changes" the directive targets.
  - No global/consistent transition spec — easing/duration is ad-hoc per call site; only onboarding uses tuned `tween` specs.

---

4. Look-and-feel / polish gaps (from video-analysis + UI-coverage audit)

From `.lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md` and `docs/qa/2026-06-25-ui-coverage-audit.md`. **Honest split:** several headline video issues are *functional* (search returns zero results, provider-set mismatch) and are OUT of scope for a UI-polish cycle — they belong to the separate search/provider-resolution fix track. The **look-and-feel / visual** subset that THIS cycle owns:

- **(video #5, HIGH) No loading indicator and no empty-state on search results** — pure blank/black body for ~25 s, then a full-screen error; the "prince" query stays blank with no state at all. This is the single biggest *visual* defect: missing `Loading` (skeleton/spinner) and `Empty` ("No results") branches → perceived hang + abrupt jump to Error. (video lines 52-59)
- **(video #4, HIGH) Provider chips render raw ids** ("torrentdownloads", "archiveorg", "kinozal", "yts") instead of display labels — same class as the §6.L 60th `displayLabel()` underscore bug. Look-and-feel/correctness. (video lines 43-50)
- **(video #8, MEDIUM) mDNS-discovered API shows bare `192.168.0.107:8443`** with no friendly name — cosmetic formatting. (video lines 79-86)
- **(video #6, MEDIUM) "4 providers available" copy** wrong vs ~12 actual — first-run copy polish. (video lines 61-68)
- **Inconsistent spacing/typography/color application:** the audit notes provider labels rendered inconsistently across surfaces (input chips vs results chips vs onboarding "All set"), and the theme tokens (`AppSpaces`/`AppTypography`) are applied ad-hoc — there is no enforced "one token, every surface" rule, which is precisely the §11.4.162 "elements MUST NOT overlap, labels MUST NOT overlay labels, consistent spacing/type" requirement.
- **Coverage gap that blocks "cover with tests":** UI-coverage audit rates only **~45% of screens** with a behavior-asserting

UI test, **~25% WEAK** (`Class.forName` reachable-only), **~30% GAP**. There is **no visual-regression layer** at all — so today there is nothing that would catch a theming/transition/overlap regression. (audit §1, §5)

---

5. Phased, parallelizable plan — 5 waves

Each wave is scoped to disjoint files where possible so they can run as parallel agents. Every UI change MUST ship light+dark variants and a §6.AB.3 falsifiability rehearsal per new test (§11.4.162 + Sixth/Seventh Law).

Wave A — OpenDesign integration + token source-of-truth (FOUNDATION; gate for B/C)

- Install OpenDesign MCP for the dev agents (`od mcp install …`); document in `docs/CODEGRAPH.md`-style guide `docs/opendesign-integration.md`.
- Author/adopt a **Lava OpenDesign design system** (`tokens.css` + `DESIGN.md`) carrying light+dark palette (seeded from brand red `0xFFB3261E`), full type scale, spacing scale, component tokens. Commit `tokens.css` as tracked source-of-truth.
- Add a **`tokens.css` → Compose codegen/sync step** that (re)generates `Colors.kt` / `AppColors` defaults, `AppTypography`, `AppSpaces`, `AppShapes` from the CSS tokens, plus a `scripts/check-opendesign-token-sync.sh` parity gate (CSS token ↔ Kotlin token; fails on drift — §6.A real-binary-contract discipline).
- **Deliverables:** `design/lava-opendesign/tokens.css` + `DESIGN.md`; codegen script; parity check + hermetic test; `libs.versions.toml` unchanged (OpenDesign is MCP/CSS, not a Gradle dep — stated honestly in the doc).
- **Operator input:** authored-vs-adopted design system; brand font choice.

Wave B — Shared screen-transition system (the "smooth + stunning" core)

- Add a **global transition spec** in `core/navigation`: a tuned `MotionScheme` (e.g. `enterSpec`/`exitSpec` as `tween`/spring with `FastOutSlowInEasing`, durations ~300/250ms) so every destination shares one tasteful timing, replacing ad-hoc per-call easing.
- **Replace the `Default` (no-transition) destinations** — searchInput→results, results→topic top-level — with directional slide+fade (forward/back aware), matching the onboarding quality bar.
- Introduce **shared-element transitions** (`SharedTransitionLayout` + `sharedBounds`/`sharedElement`) for the

highest-value morphs: search-result row → topic detail (poster/title), provider card → provider config. Tasteful, ≤350ms, reduced-motion aware.
- Honour `prefers-reduced-motion` / animator-duration-scale (disable/shorten when the OS animation scale is 0) — accessibility, not jank.
- **Deliverables:** `NavigationAnimations` extended with shared `MotionScheme` presets; per-destination reassignment in `app/.../MobileNavigation.kt`; shared-element wiring in search_result/topic + provider_config.

Wave C — Per-feature look-and-feel polish via OpenDesign tokens (light+dark)

- Sweep every `feature/*` screen to consume `AppTheme.colors/typography/spaces/shapes` (OpenDesign-sourced after Wave A) instead of ad-hoc literals; enforce consistent spacing/type.
- **Fix the visual defects this cycle owns:** add **Loading skeleton + Empty state** to `feature/search_result` (video #5); route provider chips through `displayLabel()`/`displayName` (video #4); friendly mDNS API name + remote-vs-network label (video #8); correct provider-count copy (video #6).
- Add a brand font (Wave A asset) to `AppTypography`.
- Verify **no overlap / no label collision / no font collision** per §11.4.162 across all screens, light+dark.
- **Deliverables:** polished screens; search Loading/Empty composables; label fixes; brand-font wiring.

Wave D — Visual-regression + transition + theming test infrastructure

- Stand up a **screenshot/visual-regression harness** (Roborazzi or Paparazzi — JVM, no device, fits Local-Only CI/CD) producing per-pixel/perceptual-diff PASS/FAIL golden images, **light AND dark**, for every design-system component + key screens. This is the §11.4.162 mandated "visual regression test" layer that does not exist today.
- **Transition tests:** Compose UI tests asserting the new transitions actually run (enter/exit progress, shared-element presence) and reduced-motion path; falsifiability per §6.AB.3 (break the spec → test fails).
- **Theming tests:** extend `PaletteContractTest` to assert OpenDesign-token parity + contrast (WCAG, §11.4.107) for light+dark; **overlap/collision assertions** (§11.4.162: no node bounds overlap, no label-over-label).
- **Deliverables:** Roborazzi/Paparazzi golden suite (light+dark); transition Compose UI tests; extended palette/contrast/overlap tests; `scripts/ci.sh` wiring.

Wave E — Close the UI-coverage gaps the audit named (concurrent with D)

- Per `docs/qa/2026-06-25-ui-coverage-audit.md §5`: upgrade the 5 WEAK `Class.forName` reachable challenges (C31–C35) to behavior+visual tests, and fill the GAP screens (account, main, search_input chips, credentials_manager) — now also screenshot-covered via the Wave D harness.
- **Deliverables:** rewritten C31–C35; new behavior+screenshot challenges for gap screens.

Execution note (§6.X-debt / §6.AH): the rendered-UI Challenge + screenshot golden RUNS are gated by the standing darwin/arm64 emulator/KVM gap. Roborazzi/Paparazzi are JVM (Robolectric-backed) so the *visual-regression* layer runs host-side without an emulator — a real advantage. Full on-device Challenge execution remains operator-run on a Linux x86_64+KVM or container/VM gate-host. State this honestly in each test KDoc.

---

6. What needs operator input (decisions before Wave A starts)

1. **OpenDesign design system:** author a bespoke Lava-branded system, or adopt+rebrand one of the 142 shipped systems? (§11.4.74 extend-don't-reimplement favors authoring a Lava system that extends a base.)
2. **Brand assets:** the canonical brand palette source + a **brand font** (none exists today — all `FontFamily.Default`). Needed to seed `tokens.css` + `AppTypography`.
3. **`tokens.css` as tracked source-of-truth:** confirm committing it + the generated Compose tokens, with the parity gate as the contract.
4. **OpenDesign account/license:** OpenDesign 0.7.0 is open-source/local-first (MCP server + CLI), so **no paid account expected** — but confirm `od` CLI availability on the gate host.
5. **Visual-regression tool:** Roborazzi vs Paparazzi (both JVM/local-CI-compatible; Roborazzi integrates with Compose UI tests, Paparazzi is render-only). Recommendation: **Roborazzi** (reuses the existing Compose test wiring).
6. **Scope confirmation:** the video's CRITICAL items (#1 zero-results, #2/#3 provider-set mismatch) are **functional bugs**, not look-and-feel — confirm they stay in the separate search-fix track and this cycle owns the *visual/transition* subset (#4, #5, #6, #8) + the global polish.

---

7. Bottom line

- **OpenDesign: ABSENT** in Lava (present only in `panoptic/` constitution copies) → §11.4.162 non-compliant; integration is a real deliverable. It is an **MCP/CSS-token tool, not a Gradle/Kotlin library** — integration = adopt an OpenDesign design system + generate the existing Compose token layer from its `tokens.css`.
- **Theming: mature but hand-coded.** Full light+dark, 8 palettes + Material You, real token classes (`AppColors`/`AppTypography`/`AppSpaces`/…) — but literal-sourced, **no brand font**, **no visual-regression tests**.
- **Transitions: a system exists but is under-used.** Onboarding is polished (320ms slide+fade); bottom-nav slides; but **the top-level search→results→topic path uses `Default` = no transition**, and **no shared-element transitions exist anywhere**.
- **Polish gaps owned by this cycle:** missing search Loading/Empty states (blank screens), raw provider-id labels, raw IP display, wrong provider-count copy, inconsistent token application, ~55% of screens lacking a behavior-asserting UI test and 0% visual-regression coverage.
- **Plan:** 5 waves — (A) OpenDesign integration + token source-of-truth, (B) shared/stunning transition system incl. shared-elements, (C) per-feature polish + visual-defect fixes via OpenDesign tokens (light+dark), (D) visual-regression + transition + theming test infra (Roborazzi/Paparazzi), (E) close audit UI-coverage gaps. Waves B/D/E parallelize after A; C depends on A.

Sources (OpenDesign): nexu-io/open-design, QUICKSTART.md, design-systems/README.md — (GitHub direct WebFetch blocked by network policy; facts via WebSearch result snippets, 2026-06-25).